

HDOC Day-30

Menu-Driven Array Operations in C

Problem Statement

Write a menu-driven C program to: (1) sort array elements using Bubble Sort and perform Binary Search, (2) perform insertion and deletion at a valid position, (3) rotate array elements to the right by k positions, and (4) exit.

Algorithm

1. Start the program and repeatedly display the menu until Exit is selected.
2. For each selected operation, read a positive array size and the array elements.
3. Choice 1: Use Bubble Sort to arrange elements in ascending order. Read the search key and use Binary Search on the sorted array. Display whether the element is found and its position.
4. Choice 2: Ask whether the user wants insertion or deletion. For insertion, validate position from 1 to n+1, shift elements right, and insert the new value. For deletion, validate position from 1 to n, remove the element, and shift remaining elements left.
5. Choice 3: Read k, reduce it using $k \% n$, and rotate the array right by k positions.
6. Choice 4: Display an exit message and terminate. For any other menu choice, display an error message.

Test-Case Table

TC	Choice / Purpose	Input	Expected Output
1	Bubble sort + successful binary search	n=5; array: 5 1 4 2 3; key=4	Sorted: 1 2 3 4 5; 4 found at position 4
2	Binary search - not found	n=5; array: 9 7 5 3 1; key=6	Sorted: 1 3 5 7 9; 6 not found
3	Insertion	n=4; array: 10 20 30 40; insert 25 at position 3	10 20 25 30 40
4	Deletion	n=4; array: 10 20 30 40; delete position 2	Deleted: 20; array: 10 30 40
5	Right rotation	n=5; array: 1 2 3 4 5; k=2	4 5 1 2 3
6	Boundary rotation	n=5; array: 1 2 3 4 5; k=5	1 2 3 4 5
7	Invalid insertion position	n=3; array: 1 2 3; first position=5, then position=4; value=9	Shows Invalid position; then 1 2 3 9
8	Invalid menu choice	choice=7	Invalid choice. Please enter 1 to 4.
9	Exit	choice=4	Exiting program.

C Program

Save the following program as hdoc30.c. The code below was compiled successfully using C99.

```
#include <stdio.h>
#define MAX 100

void sortAndBinarySearch(void);
void insertionDeletion(void);
void rotateRight(void);
void readArray(int a[], int *n);
void printArray(int a[], int n);

int main(void)
{
    int choice;
    do {
        printf("\n--- ARRAY OPERATIONS MENU ---\n");
        printf("1. Bubble Sort and Binary Search\n");
        printf("2. Insertion and Deletion\n");
        printf("3. Rotate array to the right by k positions\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: sortAndBinarySearch(); break;
            case 2: insertionDeletion(); break;
            case 3: rotateRight(); break;
            case 4: printf("Exiting program.\n"); break;
            default: printf("Invalid choice. Please enter 1 to 4.\n");
        }
    } while (choice != 4);
    return 0;
}

void readArray(int a[], int *n)
{
    int i;
    do {
        printf("Enter number of elements (1-%d): ", MAX);
        scanf("%d", n);
        if (*n <= 0 || *n > MAX)
            printf("Invalid size. Enter a value from 1 to %d.\n", MAX);
    } while (*n <= 0 || *n > MAX);

    printf("Enter %d elements:\n", *n);
    for (i = 0; i < *n; i++) scanf("%d", &a[i]);
}

void printArray(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}

void sortAndBinarySearch(void)
{
    int a[MAX], n, i, j, temp, key, low, high, mid, found = -1;
    readArray(a, &n);

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
```

```

        if (a[j] > a[j + 1]) {
            temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
        }
    }
}

printf("Sorted array: ");
printArray(a, n);
printf("Enter element to search: ");
scanf("%d", &key);

low = 0; high = n - 1;
while (low <= high) {
    mid = (low + high) / 2;
    if (a[mid] == key) { found = mid; break; }
    if (a[mid] < key) low = mid + 1;
    else high = mid - 1;
}

if (found != -1) printf("Element %d found at position %d in the sorted array.\n", key, found + 1);
else printf("Element %d not found in the array.\n", key);
}

```

```

void insertionDeletion(void)
{
    int a[MAX], n, op, pos, value, i;
    readArray(a, &n);

    printf("1. Insert an element\n2. Delete an element\nEnter operation: ");
    scanf("%d", &op);

    if (op == 1) {
        if (n == MAX) { printf("Array is full. Insertion is not possible.\n"); return; }
        do {
            printf("Enter insertion position (1 to %d): ", n + 1);
            scanf("%d", &pos);
            if (pos < 1 || pos > n + 1) printf("Invalid position.\n");
        } while (pos < 1 || pos > n + 1);
        printf("Enter value to insert: "); scanf("%d", &value);
        for (i = n; i >= pos; i--) a[i] = a[i - 1];
        a[pos - 1] = value; n++;
        printf("Array after insertion: "); printArray(a, n);
    } else if (op == 2) {
        do {
            printf("Enter deletion position (1 to %d): ", n);
            scanf("%d", &pos);
            if (pos < 1 || pos > n) printf("Invalid position.\n");
        } while (pos < 1 || pos > n);
        value = a[pos - 1];
        for (i = pos - 1; i < n - 1; i++) a[i] = a[i + 1];
        n--;
        printf("Deleted element: %d\n", value);
        printf("Array after deletion: "); printArray(a, n);
    } else {
        printf("Invalid operation.\n");
    }
}

```

```

void rotateRight(void)
{
    int a[MAX], n, k, i, step, last;
    readArray(a, &n);
    do {
        printf("Enter k (0 or positive): ");

```

```

scanf("%d", &k);
if (k < 0) printf("k must not be negative.\n");
} while (k < 0);

k = k % n;
for (step = 0; step < k; step++) {
    last = a[n - 1];
    for (i = n - 1; i > 0; i--) a[i] = a[i - 1];
    a[0] = last;
}
printf("Array after right rotation: "); printArray(a, n);
}

```

Outputs

Successful compilation + Choice 1 (Bubble Sort and Binary Search)

```

(kali㉿kali)-[~/Desktop]
$ vi hdoc30.c

(kali㉿kali)-[~/Desktop]
$ gcc hdoc30.c -o hdoc30 -lm

```

```

(kali㉿kali)-[~/Desktop]
$ ./hdoc30

— ARRAY OPERATIONS MENU —
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 1
Enter number of elements (1-100): 5
Enter 5 elements:
5 1 4 2 3
Sorted array: 1 2 3 4 5
Enter element to search: 4
Element 4 found at position 4 in the sorted array.

```

```

— ARRAY OPERATIONS MENU —
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 1
Enter number of elements (1-100): 5
Enter 5 elements:
9 7 5 3 1
Sorted array: 1 3 5 7 9
Enter element to search: 6
Element 6 not found in the array.

```

Choice 2 (Insertion)

```

— ARRAY OPERATIONS MENU —
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 2
Enter number of elements (1-100): 4
Enter 4 elements:
10 20 30 40
1. Insert an element
2. Delete an element
Enter operation: 1
Enter insertion position (1 to 5): 3
Enter value to insert: 25
Array after insertion: 10 20 25 30 40

```

Choice 2 (Deletion)

```

— ARRAY OPERATIONS MENU —
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 2
Enter number of elements (1-100): 4
Enter 4 elements:
10 20 30 40
1. Insert an element
2. Delete an element
Enter operation: 2
Enter deletion position (1 to 4): 2
Deleted element: 20
Array after deletion: 10 30 40

```

Choice 2 (Invalid insertion position)

```

— ARRAY OPERATIONS MENU —
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 2
Enter number of elements (1-100): 3
Enter 3 elements:
1 2 3
1. Insert an element
2. Delete an element
Enter operation: 1
Enter insertion position (1 to 4): 5
Invalid position.
Enter insertion position (1 to 4): 4
Enter value to insert: 9
Array after insertion: 1 2 3 9

```

Choice 3 (Right Rotation)

```
— ARRAY OPERATIONS MENU —
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 3
Enter number of elements (1-100): 5
Enter 5 elements:
1 2 3 4 5
Enter k (0 or positive): 2
Array after right rotation: 4 5 1 2 3
```

```
— ARRAY OPERATIONS MENU —
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 3
Enter number of elements (1-100): 5
Enter 5 elements:
1 2 3 4 5
Enter k (0 or positive): 5
Array after right rotation: 1 2 3 4 5
```

Invalid input/menu + Choice 4 Exit

```
— ARRAY OPERATIONS MENU —
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 7
Invalid choice. Please enter 1 to 4.
```

```
— ARRAY OPERATIONS MENU —
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 4
Exiting program.
```

